

# JAVA INTERVIEW PREPARATION

CHAPTER 70

TRIES, PREFIX INDEXES,  
AND STRING SEARCH

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

# Tries, Prefix Indexes, and String Search

Senior / Lead Java Developer Reading Chapter

A trie organizes keys by shared prefixes. Instead of storing each string as one indivisible comparison key, it consumes one text unit at a time and turns prefixes into explicit paths. This makes lookup proportional to key length and supports prefix enumeration naturally. It also creates costs that simple Big-O summaries hide: node fan-out, object overhead, Unicode semantics, normalization, ranking, deletion, and snapshot consistency.

Tries appear in autocomplete, routing tables, dictionaries, spell support, configuration paths, IP-prefix matching, and multi-pattern search. They are not automatically better than hash maps. The correct choice depends on whether prefix operations are first-class, how large and sparse the alphabet is, and whether memory layout can be controlled.

## Prefixes become paths

```
keys: car, card, care, cat

root
 |
 c
 |
  a
 / \
r   t*
*
 | \
d* e*
```

\* marks the end of a complete key

The path `c -> a -> r` exists for `car`, `card`, and `care`. A terminal marker distinguishes a complete key from a prefix. Without it, the structure cannot tell whether `car` was inserted or merely exists as the shared path toward longer keys.

If key length is `L`, lookup and insertion are  $O(L)$  under constant-time child access. That does not mean independent of input size in every implementation: a node with children stored in a list may require scanning its fan-out, and a tree map adds a logarithmic child lookup. Complexity must name both key units and child representation.

## A clear Java model

For a known small alphabet, an array per node is simple and fast. For general text, allocating a huge child array at every node is wasteful, so maps are more flexible.

```

final class Trie {
    private static final class Node {
        final Map<Integer, Node> children = new HashMap<>();
        boolean terminal;
    }

    private final Node root = new Node();

    void add(String key) {
        Objects.requireNonNull(key);
        Node node = root;
        PrimitiveIterator.OfInt units = key.codePoints().iterator();
        while (units.hasNext()) {
            int unit = units.nextInt();
            node = node.children.computeIfAbsent(unit, ignored -> new Node());
        }
        node.terminal = true;
    }

    boolean contains(String key) {
        Node node = findNode(key);
        return node != null && node.terminal;
    }

    boolean containsPrefix(String prefix) {
        return findNode(prefix) != null;
    }
}

```

This example chooses Unicode code points as edge labels. That is already safer than treating every UTF-16 `char` as a user character, but code points are not the same as grapheme clusters. The choice must follow the domain.

## Text identity must be defined first

Should `Cafe`, `cafe`, and `café` be distinct? Should composed `é` equal `e` plus a combining accent? Should a search ignore case according to English rules, Turkish rules, or locale-independent identifiers? A trie cannot repair an undefined identity policy.

```

raw input
  |
  v
validate -> normalize -> case policy -> choose units -> trie path

```

Normalize keys before insertion and lookup using the same pipeline. Unicode normalization such as NFC can make canonically equivalent sequences share a representation. Case folding is more complex than calling `toLowerCase()` without a locale. For protocol identifiers, use the protocol's defined ASCII or locale-independent rules. For human language, search behavior may require a specialized collation or search library.

Code-point tries preserve supplementary characters as one edge but may still split a visible grapheme. Grapheme-aware behavior can use boundary analysis before indexing, with greater complexity and different matching semantics. Byte-oriented tries are appropriate when keys are explicitly UTF-8 byte sequences or network prefixes, but byte prefixes may not correspond to user-visible text boundaries.

## Searching and prefix enumeration

Exact lookup follows the path and then checks the terminal marker. Prefix lookup follows the path but does not require terminal status. Autocomplete first locates the prefix node, then enumerates terminal descendants.

```

List<String> complete(String prefix, int limit) {
    if (limit < 0) throw new IllegalArgumentException("limit");
    Node start = findNode(prefix);
    if (start == null || limit == 0) return List.of();

    List<String> output = new ArrayList<>(Math.min(limit, 16));
    StringBuilder path = new StringBuilder(prefix);
    collect(start, path, output, limit);
    return output;
}

private void collect(Node node, StringBuilder path,
                    List<String> output, int limit) {
    if (output.size() == limit) return;
    if (node.terminal) output.add(path.toString());

    for (var entry : node.children.entrySet()) {
        if (output.size() == limit) break;
        int oldLength = path.length();
        path.appendCodePoint(entry.getKey());
        collect(entry.getValue(), path, output, limit);
        path.setLength(oldLength);
    }
}

```

The backtracking invariant is that `path` represents exactly the current root-to-node path. Save its UTF-16 length before appending and restore that length afterward. Because `HashMap` iteration is unspecified, result order is unspecified. Use sorted child maps, sorted keys, or an explicit ranking policy if order matters.

Enumeration cost is at least proportional to emitted content. Saying autocomplete is  $O(\text{prefix length})$  ignores traversal below the prefix and output size. A limit protects latency and memory but does not automatically find the best results.

## Ranking autocomplete results

Real autocomplete is usually ranked by popularity, recency, personalization, edit distance, or business rules. Lexicographic DFS is only one order.

One design stores a small top-k suggestion list at every prefix node:

```

prefix "ca" node
cached suggestions:
1. camera    score 920
2. car       score 870
3. calendar  score 650

```

Lookup becomes fast because it reads the cached list after following the prefix. Writes become more expensive: inserting or changing one key may update every prefix on its path. Popularity changes can create a write-heavy hot path. Alternatives compute ranking at query time, update caches asynchronously, or use a search index designed for ranked prefix retrieval.

Scores need versioning and tie-breakers. Personalized results should not be written into globally shared nodes. Offensive or unauthorized suggestions require filtering before they are cached or returned. Product correctness extends beyond the data structure.

## Deletion and pruning

Deleting a key first clears its terminal marker. Nodes can be pruned upward only while they are nonterminal and have no children.

keys before: car, card

delete card:  
clear terminal at d  
remove d because it has no children  
keep r because car is still terminal

```
boolean remove(String key) {
    int[] units = key.codePoints().toArray();
    return remove(root, units, 0).removed();
}

private record Removal(boolean removed, boolean prune) {}

private Removal remove(Node node, int[] units, int index) {
    if (index == units.length) {
        if (!node.terminal) return new Removal(false, false);
        node.terminal = false;
        return new Removal(true, node.children.isEmpty());
    }

    Node child = node.children.get(units[index]);
    if (child == null) return new Removal(false, false);
    Removal result = remove(child, units, index + 1);
    if (result.prune()) node.children.remove(units[index]);
    boolean prune = !node.terminal && node.children.isEmpty();
    return new Removal(result.removed(), prune);
}
```

Recursive deletion has stack depth equal to key length. Extremely long untrusted keys require validation or an iterative path stack. If nodes cache counts or top suggestions, deletion must update that metadata consistently.

## Child representation trade-offs

An array gives  $O(1)$  child access and good predictable indexing but allocates a slot for every possible symbol at every node. It is effective for digits, lowercase ASCII, DNA bases, or a compressed alphabet.

A hash map stores only existing children and offers expected  $O(1)$  access, but each map and entry adds substantial overhead. A sorted map supports ordered enumeration with  $O(\log \text{ degree})$  access. Small-node optimization can store a few children in compact arrays and promote to a map above a threshold.

Dense fixed array node

[a][b][c]...[z]  
fast, predictable  
many empty slots

Sparse map node

{ 'a' -> node, 't' -> node }  
memory follows actual degree  
more objects and indirection

Object-per-node tries can consume far more memory than the key text. Measure retained size, not just node count. Primitive collections, packed arrays, string tables, and immutable serialized tries can greatly improve locality.

A rough memory estimate prevents surprises. One million keys of average length 20 do not necessarily mean only 20 million character units. If prefix sharing leaves 12 million nodes and each node plus child container averages even 40 bytes, retained memory approaches 480 MB before payload, ranking lists, hash-table slack, and the key-to-ID mapping. Measure with realistic data because prefix sharing and fan-out distribution determine the result.

Immutable packed representations can assign every node an integer and store edge labels, child offsets, and terminal bits in primitive arrays. They trade convenient mutation for dense layout, fast publication, and memory-mapped serving. This is often a good fit when an index is rebuilt periodically and queried frequently.

## Compressed tries and radix trees

A radix tree compresses chains of single-child nodes into string-labeled edges.

|                   |            |
|-------------------|------------|
| ordinary trie     | radix tree |
| c -> a -> r -> d* | "car"      |
|                   | \          |
| e*                | "d"* "e"*  |

Lookup compares an edge label segment rather than one unit. Compression reduces nodes and improves locality, but insertion and deletion may split or merge edges. Patricia tries apply related compression, commonly to bit strings such as network prefixes.

For IP routing, longest-prefix match selects the most specific route whose bit prefix matches the address. This is not ordinary lexicographic autocomplete. IPv4 and IPv6 widths, CIDR validation, and route precedence must be modeled explicitly.

## Ternary search tries

A ternary search trie stores one symbol per node plus low, equal, and high links. It combines character comparison with prefix progression.

```
      'm'
     /  |  \
  lower next higher
```

It can use less space than a large child array while supporting ordered traversal. Its performance depends on symbol-tree balance and key distribution. It is one design point, not a universally superior trie.

## Multi-pattern search with Aho-Corasick

Searching a text independently for thousands of patterns repeats work. Aho-Corasick builds a trie of patterns and adds failure links. When the current path cannot continue, a failure link jumps to the longest suffix that is also a trie prefix.

```
patterns: he, she, his, hers

trie edges identify prefixes
failure links reuse suffix knowledge
output sets report every pattern ending at a state
```

After preprocessing, scanning is linear in text length plus reported matches, subject to alphabet and representation costs. This is useful for signature matching, content classification, and log rules. It differs from a suffix tree or suffix array, which indexes substrings of one text rather than a set of patterns.

Production matchers must bound pattern count, total pattern length, output volume, and adversarial match density. Returning every occurrence of very common patterns can dominate scan time and memory even when state transitions are linear.

## Tries versus other indexes

A hash map is usually simpler and more memory-efficient for exact lookup only. A sorted array offers compact storage and binary search, with prefix ranges located by lower bounds. A [TreeMap](#) supports ordered updates and range views. A trie provides direct prefix navigation and can share prefix storage, but object overhead may exceed saved characters.

Search engines often use finite-state transducers, term dictionaries, inverted indexes, n-grams, or specialized completion structures. Database **LIKE** `'prefix%'` may use a suitable B-tree index depending on collation and query plan. Do not build an in-process trie merely because the requirement contains the word "search."

| requirement                   | likely starting point            |
|-------------------------------|----------------------------------|
| exact key lookup              | HashMap                          |
| ordered key/range             | TreeMap or sorted array          |
| high-volume prefix completion | trie/radix/specialized index     |
| substring search              | suffix/n-gram/inverted technique |
| many patterns through text    | Aho-Corasick                     |

## Concurrency and snapshots

A mutable trie update touches every node along a path and may split, prune, or update ranking metadata. Concurrent readers cannot safely observe partially coordinated changes merely because child maps are concurrent. Define an atomic publication boundary.

One approach builds an immutable trie snapshot and replaces the root reference after construction. Readers obtain consistent lock-free snapshots; writes allocate new path nodes through structural sharing or rebuild in batches. Another approach uses locks by subtree, but lock ordering, ranking metadata, deletion, and prefix enumeration complicate correctness.

Autocomplete services often separate an offline index build from online serving. Publish versioned artifacts, warm them before traffic, retain the prior version for rollback, and expose index version and age in telemetry. This turns a complex concurrent mutation problem into controlled snapshot replacement.

The build pipeline should be deterministic. Given the same normalized input and ranking snapshot, it should produce the same ordered suggestions and checksums. Validate node counts, terminal counts, duplicate handling, banned-term filtering, and a sample query suite before publication. A canary can load the new artifact and compare latency, memory, and result quality before the root version is switched globally.

Fuzzy completion adds another dimension. One approach traverses trie paths while maintaining a dynamic-programming edit-distance row and prunes branches whose minimum possible distance exceeds the threshold. This can be effective for small edit limits, but worst-case work can expand dramatically. Product search systems often combine prefix indexes with n-gram candidates, phonetic signals, and ranking rather than relying on one trie traversal.

## Production and security considerations

Limit key length, prefix length, result count, and traversal work. A request for an empty prefix can mean "enumerate the entire dictionary" unless explicitly rejected. High-cardinality tenant data needs isolation and quotas. Normalization expansion and malformed surrogate input need defined handling.

Measure node count, edge count, retained bytes, average and maximum depth, fan-out distribution, prefix-query latency, result truncation, index build time, publication time, and stale-version age. Query logs may contain sensitive text; avoid leaking full prefixes into unprotected telemetry.

## Testing trie behavior

Test empty key policy, a key that is a prefix of another, duplicates, deletion with shared paths, missing keys, supplementary code points, combining forms, normalization, case rules, very long keys, and output limits.

Useful properties include:

- every inserted normalized key is found;
- removing one key does not remove another sharing its prefix;

- every completion starts with the requested normalized prefix;
- completion contains no key not present in the reference set;
- radix compression preserves exact and prefix results;
- snapshot publication never exposes a partially built version.

Compare randomized operations against a reference `Set<String>` plus filtered/sorted prefix queries. Use explicit deterministic ranking or avoid asserting order.

## Common senior-level traps

- Forgetting terminal markers and confusing prefixes with complete keys.
- Calling lookup  $O(1)$  instead of  $O(\text{key length})$ .
- Using `char` while claiming Unicode-character semantics.
- Applying different normalization or case policy at insert and lookup.
- Assuming hash-map child iteration is lexicographic.
- Reporting autocomplete as  $O(\text{prefix length})$  while ignoring output traversal.
- Deleting shared nodes needed by another key.
- Ignoring object overhead and sparse fan-out.
- Mutating ranking metadata without an atomic snapshot strategy.
- Using a trie for substring search without a matching index design.
- Allowing empty-prefix enumeration or unbounded result output.

## A strong interview answer

A trie turns each normalized key into a path of chosen text units, with a terminal marker distinguishing complete keys. Lookup is proportional to key length plus child-access cost, and prefix enumeration additionally costs the visited/output subtree. Arrays suit small fixed alphabets; maps suit sparse general alphabets; radix compression reduces single-child chains. Before implementation I define normalization, case, Unicode unit, ranking, and deletion semantics. In production I measure retained memory, bound completion output, and prefer immutable versioned snapshots when concurrent readers need coherent results.

## Chapter summary

Tries make prefixes explicit and therefore support prefix lookup and enumeration naturally. Their correctness depends on terminal markers and a consistent text-identity pipeline. Their practical performance depends on child representation, compression, ranking, memory layout, and output bounds. Radix trees compress paths, longest-prefix tries serve routing, and Aho-Corasick extends a pattern trie with failure links for multi-pattern scanning. Production systems often gain more from snapshot publication and careful product semantics than from the basic lookup loop.



# Revision Notes

- A trie maps normalized key units onto root-to-node paths.
- Terminal markers distinguish complete keys from prefixes.
- Lookup is  $O(L)$  only when child access is treated as  $O(1)$ ; name the unit and representation.
- Code points are not UTF-16 code units and still are not grapheme clusters.
- Use the same validation, normalization, and case policy for writes and reads.
- Prefix completion costs prefix lookup plus subtree/output traversal.
- [HashMap](#) children do not provide lexical enumeration order.
- Cached top-k suggestions accelerate reads but amplify writes and invalidation.
- Delete by clearing terminal state, then prune only empty nonterminal ancestors.
- Fixed arrays suit small alphabets; maps suit sparse fan-out.
- Object-per-node tries can have high retained memory and poor locality.
- Radix trees compress chains into substring-labeled edges.
- Longest-prefix matching differs from lexicographic completion.
- Aho-Corasick adds failure links for scanning many patterns together.
- Exact lookup alone often favors a hash map; ordered prefix ranges may favor sorted data.
- Immutable versioned roots provide coherent concurrent snapshots.
- Bound key length, prefix traversal, output size, and index-build resources.

Likely interview question: Why does a trie need an end-of-word marker?

Because a path may exist only as a prefix of a longer key. The marker distinguishes an inserted key such as [car](#) from the same path created only by [card](#) and [care](#).

Likely interview question: Is trie lookup  $O(1)$ ?

No. It consumes the key, so it is  $O(L)$  in the number of indexed units, assuming constant-time child access. Map or sorted-child lookup, normalization, and cache behavior add practical cost.

Likely interview question: How would you design autocomplete at scale?

Define normalization and ranking first, use a compact immutable prefix index, cache a bounded top-k list at useful nodes or compute ranking through a search platform, publish versioned snapshots, enforce tenant and result limits, and measure freshness plus truncation.

Likely interview question: Trie versus TreeMap for prefixes?

A trie navigates directly by prefix and can share prefix structure. A [TreeMap](#) is simpler for ordered dynamic keys and can retrieve a prefix range through lower boundaries. Choose using update rate, memory, ranking, result size, and operational complexity rather than asymptotics alone.